Word Embeddings for Sentiment Classification: Word2Vec CBOW vs GloVe vs fastText CBOW with an LSTM

Team members (student IDs): hoa25ms13299, hiep25ms13293, long25ms13310, anh25ms13296

1. Setup

Task and data. Binary sentiment classification on IMDB (25,000 train / 25,000 test reviews). The official train split was divided 80/20 into 20,000 training and 5,000 validation reviews (permutation seed 42); the official test split was used only for the final evaluation.

Controlled pipeline. One tokenizer (lowercase, letters and apostrophes, `<br />` removed), one vocabulary built from training reviews only (words with frequency >= 2; 46,321 entries including `<pad>` and `<unk>`), sequences cut or padded to 300 tokens. Only the embedding matrix changes between conditions. The classifier is one unidirectional LSTM layer (hidden size 128, one logit, `BCEWithLogitsLoss`, Adam at 1e-3, batch size 64, early stopping on validation loss with patience 2, gradient clipping at 5). Every condition was run with seeds 0, 1 and 2.

Embeddings. Word2Vec CBOW and fastText CBOW were trained on the 20,000 training reviews only (dimension 100, window 5, min_count 2, negative sampling, 10 epochs; fastText n-grams 3 to 6). GloVe is the public `glove-wiki-gigaword-100` file (Wikipedia 2014 + Gigaword 5, 6B tokens), so its corpus differs from IMDB by about three orders of magnitude (6B vs 4.6M tokens). Missing rows were initialised from N(0, 0.1^2); `<pad>` is zero.

Runs. E0 random (frozen), E1 Word2Vec, E2 GloVe, E3 fastText (frozen), E4 all three trainable; plus the extension `E3dyn` (dynamic fastText lookup for out-of-vocabulary tokens at inference). Hardware: Apple M1 Max, 32 GB, PyTorch MPS.

2. Results on the official test set (25,000 reviews, mean ± std over 3 seeds)

Run	Embedding	Layer	Accuracy	Macro-F1	ROC-AUC	Rare-subset macro-F1
E0	Random	Frozen	0.679 ± 0.048	0.677 ± 0.048	0.743 ± 0.063	0.632 ± 0.044
E1	Word2Vec CBOW	Frozen	**0.873 ± 0.004**	**0.873 ± 0.004**	**0.945 ± 0.002**	**0.836 ± 0.010**
E2	GloVe	Frozen	0.867 ± 0.003	0.867 ± 0.003	0.941 ± 0.003	0.835 ± 0.001
E3	fastText CBOW	Frozen	0.864 ± 0.001	0.864 ± 0.001	0.935 ± 0.003	0.826 ± 0.003
E3dyn	fastText + dynamic OOV lookup	Frozen	0.864 ± 0.001	0.864 ± 0.001	0.936 ± 0.003	0.830 ± 0.002
E4	Word2Vec CBOW	Trainable	0.859 ± 0.006	0.859 ± 0.006	0.932 ± 0.004	0.813 ± 0.008
E4	GloVe	Trainable	0.845 ± 0.003	0.844 ± 0.003	0.928 ± 0.005	0.789 ± 0.004
E4	fastText CBOW	Trainable	**0.866 ± 0.002**	**0.866 ± 0.002**	**0.937 ± 0.002**	0.821 ± 0.007

Precision, recall, F1 and per-seed values are in `results/metrics.csv`; aggregated values are in `results/summary.csv`. The "rare subset" is the 2,500 test reviews (10%) with the highest share of `<unk>` tokens (>= 3.5%). Figures: `results/learning_curves.png`, `results/learning_curves_trainable.png`, `results/confusion_matrices.png`, `results/confusion_matrices_trainable.png`.

Coverage and cost.

Source	Vocabulary words found	Test tokens with a real vector	Training cost
Word2Vec	46,319 / 46,319 (100%)	98.34%	16 s
fastText	46,319 / 46,319 (100%)	98.34%	50 s
GloVe	39,637 / 46,319 (85.57%)	96.46%	none (file load; ~128 MB download)

The `<unk>` rate is shared by all conditions: 1.24% of validation tokens and 1.66% of test tokens (27.9% and 54.6% of the unique word types respectively). Every LSTM has 4,749,989 parameters, of which 117,889 are trainable when the embedding is frozen (E0 to E3) and all 4,749,989 when it is trainable (E4). A frozen run took 80 to 250 s (mean 106 to 140 s) and a trainable run 65 to 166 s on MPS; peak process RSS was about

1.6 to 1.8 GB (cumulative for the whole process, so not a clean per-run figure). Runtime differences here are not meaningful: they are driven almost entirely by how many epochs early stopping allowed (for example E0 seed 1 trained for 17 epochs and took 246 s), not by the embedding source. Embedding training itself (16 s vs 50 s) is negligible next to LSTM training.

Figures



Figure 1. Training (solid) and validation (dashed) loss, frozen embeddings, three seeds.

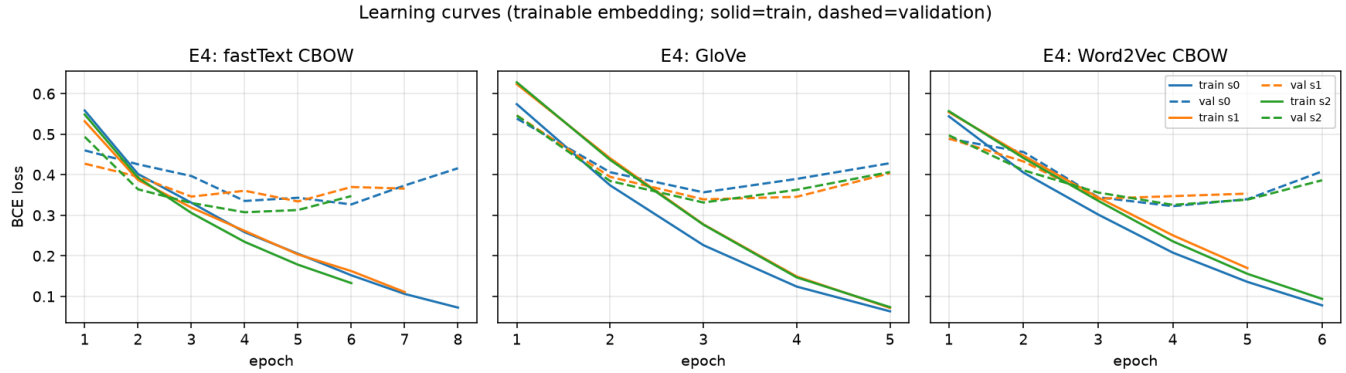


Figure 2. Training and validation loss, trainable embeddings: validation loss turns upward after epoch 3 to 6 while training loss keeps falling.

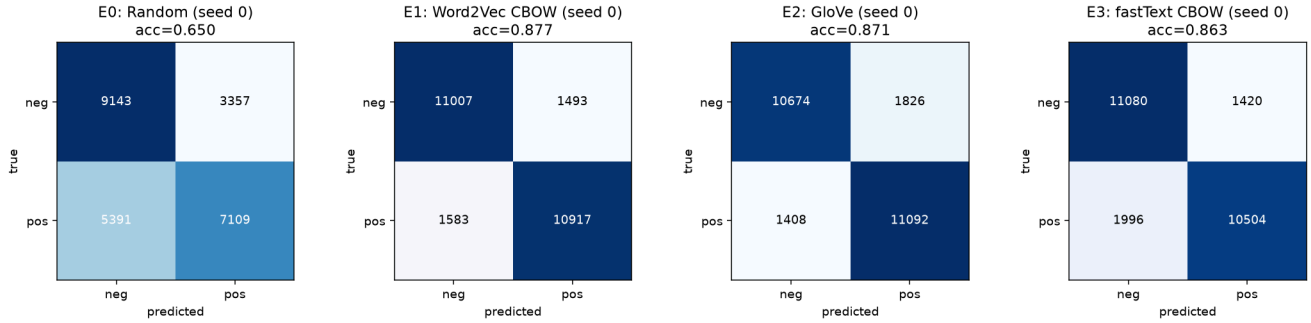


Figure 3. Confusion matrices on the official test set, frozen embeddings (seed 0).

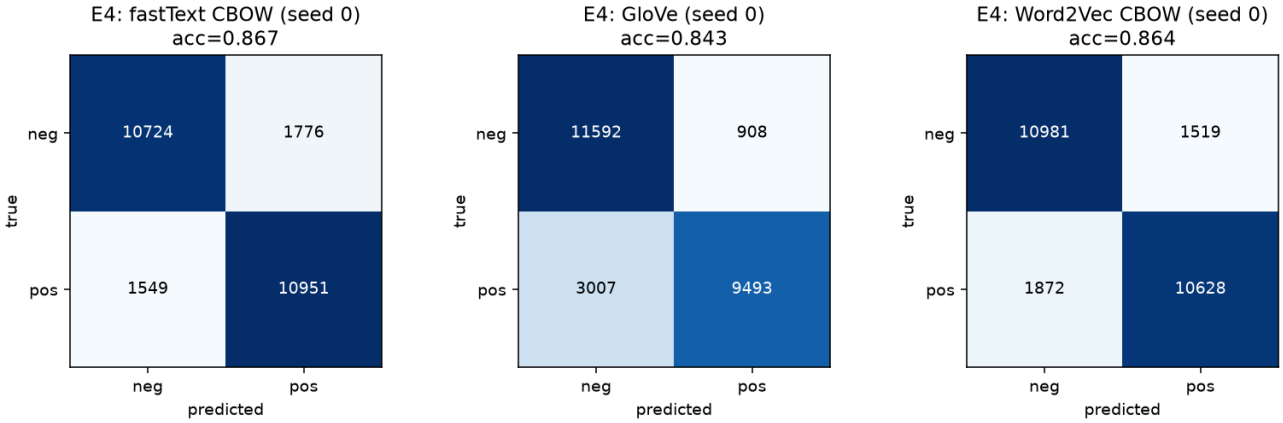


Figure 4. Confusion matrices on the official test set, trainable embeddings (seed 0).

3. Answers to the required questions

1. Which embedding gives the strongest test macro-F1, and does the difference matter across seeds? Frozen Word2Vec CBOW is highest (0.873), followed by GloVe (0.867) and fastText (0.864). Word2Vec beats both in all three seeds (by 0.6 and 0.9 points on average), so the ordering E1 > E2/E3 is consistent, but the margins are small: they are under 1 point, with a seed standard deviation of 0.3 to 0.4 points and a binomial standard error of about 0.2 points on 25,000 test reviews. GloVe and fastText cannot be separated reliably: the gap (0.7 points) comes mostly from seed 0, while seeds 1 and 2 differ by less than 0.1 point. With only three seeds, and with the embeddings themselves trained once (seed 42), seed-to-seed variation from embedding training is not captured. The one large effect is the random control: all three pretrained or in-domain embeddings are about 19 to 20 points better than E0, which is also unstable (macro-F1 0.648, 0.733, 0.651; one seed needed 17 epochs), so any semantic initialisation clearly helps a frozen embedding.

2. Does the ranking change when the embedding is trainable? Yes. With trainable embeddings the order becomes fastText (0.866) > Word2Vec (0.859) > GloVe (0.844). Fine-tuning helped fastText marginally (+0.2 points) and hurt Word2Vec (-1.4) and GloVe (-2.3). The learning curves explain why: a trainable table adds about 4.63M parameters to a 20,000-review training set, training loss falls to roughly 0.07 while validation loss rises after epoch 3 to 6, and early stopping picks a checkpoint after only 3 to 6 epochs (frozen runs used 6 to 12). Fine-tuning therefore overfits and can erode the general structure of the initial vectors; this is largest for the externally trained GloVe vectors, whose geometry has to adapt to a much smaller, differently distributed corpus. Why fastText is less affected is a hypothesis we did not test (its vectors are sums of word and n-gram vectors, which may be smoother); the data only show that it is the least harmed. Fine-tuning does not make the choice of initial vectors irrelevant, but it reduces the value of the best frozen vectors.

3. How do coverage and `<unk>` rates relate to performance? Is higher coverage always better? No. Word2Vec and fastText have identical coverage (100% of vocabulary words, 98.34% of test tokens) because they share the training text and `min_count`, and they differ by 0.9 points. GloVe has the lowest coverage (85.57% of vocabulary words, 96.46% of test tokens) yet outperforms fastText and is within 0.6 points of the best. A plausible explanation (not tested here) is that the words GloVe misses are mostly rare or domain-specific, while the frequent words that carry sentiment are covered by every source. The shared `<unk>` rate is small on the whole test set (1.66%), so it explains little overall. On the rare subset (>= 3.5% `<unk>`) macro-F1 drops by about 3 to 4 points for E1 to E3 (0.873 to 0.836, 0.867 to 0.835, 0.864 to 0.826), but this comparison is confounded because those reviews also differ in length, style and spelling. Coverage is therefore a necessary diagnostic rather than a predictor, and quality and domain match of the vectors matter more than the fraction of words found.

4. Evidence for or against the claim that fastText helps with rare words, morphology and spelling variation. The evidence is mixed and modest.

- *Supporting:* fastText returns a vector for any string. In `results/subword_demo.txt`, "terrrible" has terrible, horrible and horrendous as nearest neighbours, "amazinggg" has amazing, and "movvie" has movie, while Word2Vec has no vector for any of these. With dynamic lookup (`E3dyn`, 46,114 test types given subword vectors instead of `<unk>`), macro-F1 on the rare subset rose from 0.826 to 0.830 in all three seeds (+0.4 to +0.5 points).
- *Challenging:* the same lookup left overall test macro-F1 unchanged (0.864 vs 0.864). Frozen fastText was not better than Word2Vec or GloVe on the rare subset (0.826 vs 0.836 and 0.835). Subword similarity is also noisy: "awsome" was in the training vocabulary (so not out-of-vocabulary) and its neighbours were irksome, 'some, fulsome and loathsome, which are wrong for a misspelled "awesome".
- *Not tested:* the optional controlled spelling-variation experiment was not performed, so the spelling-robustness claim is supported only by the nearest-neighbour demonstration.
- *Caution:* fastText and Word2Vec were trained on the same 4.6M tokens, while GloVe saw 6B tokens, so GloVe's competitiveness reflects corpus size and domain as much as its objective. Subword information, training objective, corpus size, domain and coverage are all different between rows and cannot be separated by this design.

5. What kinds of reviews are misclassified by all models, and what does it reveal? 944 of 25,000 test reviews (3.8%) were misclassified by all four frozen conditions (using mean probabilities over seeds). The 20 reviews inspected (12 most confidently wrong plus 8 random, `results/error_analysis.md`) fall into these themes, with overlaps:

- *Label and text disagree (about 6 of 20):* for example test[4118] ("one of the best Kung fu movies... a masterpiece") and test[3757] ("an excellent movie... Highly Recommended: 7/10") are labelled negative; test[17942] is labelled positive but is clearly negative. These are probably noisy labels, so a ceiling on accuracy below 100%. This theme is inflated by sorting on confidence; the random subset is more representative.
- *Ironic praise or "so bad it is good" (3):* test[18896] ("the script is terrible... a veritable delight"), test[14565] ("so incredibly bad-made that it becomes funny... 10/10") and test[16048] use strongly negative vocabulary for a positive verdict. Averaged embeddings in a last-state LSTM cannot represent this inversion.
- *Concession, negation and mixed opinions (about 6):* test[18713] ("I enjoyed this series, but... let down by the sound"), test[16754] and test[7157] put the verdict in a clause that the model weighs against several negative words.
- *Long reviews (6 of 20 exceed 300 tokens):* the model reads only the first 300 tokens, so the closing verdict is cut off (test[9543], test[23038], test[8119]), although the reviews are mostly just over the limit.

- *Rare words and spelling (1 of 20):* only one example (test[9103], 5.8% `<unk>`) shows this, so rare words are not a major error source here.
- *Ambiguous or spam text (2):* test[24403] and test[23647] (a repeated paragraph). The heuristic tags in the file are only a first pass. Overall, the errors that survive all four embeddings reflect limits of word-level vectors and a one-layer last-state LSTM (no sarcasm or discourse structure, truncation) plus label noise, not missing vocabulary.

6. Which embedding would you deploy on a new-domain sentiment dataset, and what would you validate? I would start from in-domain fastText vectors if there is a few million tokens of unlabelled text from the new domain (they handle slang, spelling variation and new vocabulary without relying on an external vocabulary), and from GloVe (or other pretrained vectors) if there is little domain text. Our experiment alone does not justify choosing fastText: in the same-domain setting the three were within about 1 point, and in-domain Word2Vec was best frozen. Before deployment I would require: (a) a held-out labelled set from the target domain with several seeds and confidence intervals; (b) vocabulary coverage and `<unk>` rates measured on target text; (c) a robustness test with spelling and slang variation (not done here); (d) a comparison of frozen vs fine-tuned vectors, since fine-tuning overfit in our case; (e) a check against a stronger baseline such as a fine-tuned transformer; and (f) review of label quality and of failure cases like sarcasm, because they set the achievable ceiling.

4. Limitations

- Three seeds only; Word2Vec and fastText were trained once, so embedding-training variance is not included in the standard deviations. Differences under about 1 point should be treated as tentative.
- GloVe was not trained on IMDB, so GloVe vs Word2Vec/fastText mixes corpus size, domain and objective (see Q4). No hyperparameters were tuned for any method, by design.
- Each review is truncated to its first 300 tokens; the unidirectional LSTM uses its last hidden state.
- Training ran on Apple MPS, so results are not bit-for-bit reproducible across machines.
- Peak memory is a process-level cumulative figure. The spelling-variation set, Skip-gram, BiLSTM, attention and transfer extensions were not run. The error-analysis themes come from 20 reviews read by hand and partly selected by confidence.

5. Sources and tools

IMDB via Hugging Face Datasets (`stanfordnlp/imdb` , the dataset behind `load_dataset("imdb")`); GloVe `glove-wiki-gigaword-100` via `gensim.downloader` (Pennington et al., 2014); PyTorch, Gensim, scikit-learn, Matplotlib. The code and this draft were produced with the help of the generative coding tool Claude Code; all numbers come from the runs documented in `results/` . Commands, environment and file descriptions are in `README.md` .

8 Required experiments

Team members (student IDs): hoa25ms13299, hiep25ms13293, long25ms13310, anh25ms13296

Run	Embedding condition	Embedding layer	Purpose	Best configuration	Results
E0	Random 100-dimensional vectors	Frozen or fixed control	Measure what the LSTM learns without external semantic initialization.	Frozen embedding; LSTM 1x128, Adam 1e-3, batch 64, patience 2 best epoch 5 / 15 / 4 (seeds 0/1/2); mean val loss 0.601	**Completed - seeds 0, 1, 2** (mean ± std) Accuracy: **67.92% ± 4.77** Precision / Recall: 0.7130 ± 0.0571 / 0.6000 ± 0.0471 F1 (positive): 0.6516 ± 0.0516 Macro-F1: **0.6772 ± 0.0480** ROC-AUC: **0.7427 ± 0.0632**
E1	Word2Vec CBOW trained on IMDB training text	Frozen	Test word-level local context representations.	Frozen embedding; LSTM 1x128, Adam 1e-3, batch 64, patience 2 best epoch 8 / 6 / 7 (seeds 0/1/2); mean val loss 0.313	**Completed - seeds 0, 1, 2** (mean ± std) Accuracy: **87.29% ± 0.43** Precision / Recall: 0.8762 ± 0.0100 / 0.8687 ± 0.0078 F1 (positive): 0.8724 ± 0.0038 Macro-F1: **0.8729 ± 0.0043** ROC-AUC: **0.9448 ± 0.0017**
E2	GloVe 100-dimensional public vectors	Frozen	Test global co-occurrence representations from an external corpus.	Frozen embedding; LSTM 1x128, Adam 1e-3, batch 64, patience 2 best epoch 12 / 8 / 8 (seeds 0/1/2); mean val loss 0.319	**Completed - seeds 0, 1, 2** (mean ± std) Accuracy: **86.68% ± 0.33** Precision / Recall: 0.8690 ± 0.0101 / 0.8642 ± 0.0210 F1 (positive): 0.8664 ± 0.0056 Macro-F1: **0.8668 ± 0.0033** ROC-AUC: **0.9411 ± 0.0026**
E3	fastText CBOW trained on IMDB training text	Frozen	Test the value of character n-grams and subword information.	Frozen embedding; LSTM 1x128, Adam 1e-3, batch 64, patience 2 best epoch 6 / 5 / 6 (seeds 0/1/2); mean val loss 0.334	**Completed - seeds 0, 1, 2** (mean ± std) Accuracy: **86.40% ± 0.06** Precision / Recall: 0.8625 ± 0.0159 / 0.8666 ± 0.0228 F1 (positive): 0.8643 ± 0.0036 Macro-F1: **0.8639 ± 0.0006** ROC-AUC: **0.9352 ± 0.0025**
E4	Word2Vec, GloVe, and fastText repeated	Trainable	Measure how much task-specific fine-tuning changes the comparison.	Trainable embedding; LSTM 1x128, Adam 1e-3, batch 64, patience 2 **Word2Vec:** best epoch 4 / 3 / 4 (seeds 0/1/2); mean val loss 0.330 **GloVe:** best epoch 3 / 3 / 3 (seeds 0/1/2); mean val loss 0.342 **fastText:** best epoch 6 / 5 / 4 (seeds 0/1/2); mean val loss 0.323 **Best (lowest val loss): fastText**	**Completed - seeds 0, 1, 2 (mean ± std)** **Word2Vec** Accuracy: **85.87% ± 0.58** Precision: 0.873 ± 0.013 Recall: 0.840 ± 0.025 F1 (positive): 0.856 ± 0.008 Macro-F1: **0.859 ± 0.006** ROC-AUC: **0.932 ± 0.004** **GloVe** Accuracy: **84.48% ± 0.32** Precision: 0.890 ± 0.024 Recall: 0.788 ± 0.027 F1 (positive): 0.835 ± 0.006 Macro-F1: **0.844 ± 0.003** ROC-AUC: **0.928 ± 0.005** **fastText** Accuracy: **86.55% ± 0.24** Precision: 0.867 ± 0.006 Recall: 0.864 ± 0.013 F1 (positive): 0.865 ± 0.004 Macro-F1: **0.866 ± 0.002** ROC-AUC: **0.937 ± 0.002**

Evaluation: Official IMDB test set, 25,000 reviews; one-layer unidirectional LSTM (128 hidden units).

Selection: Best epoch chosen by validation loss (best epochs observed: 3-15); early stopping patience 2. Seeds: 0, 1, 2.

Source: results/metrics.csv (per run) and results/summary.csv. Precision, recall and positive-class F1 use label 1; threshold = 0.5.